

Hermes Outreach Dashboard - Implementation Plan

Overview

React + Vite dashboard for managing outreach leads, WhatsApp queue, follow-ups, and reports. Connects to the engine API server.

Tech Stack

- **Framework:** React 18 + Vite
 - **Language:** TypeScript
 - **Styling:** Tailwind CSS
 - **Routing:** React Router DOM v6
 - **HTTP Client:** Axios
 - **Charts:** Recharts
 - **Date handling:** date-fns
 - **Export:** PapaParse (CSV), XLSX (Excel)
 - **Utility:** clsx
-

Project Structure

```
dashboard/
├── public/
│   └── vite.svg
├── src/
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Layout.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   ├── Header.tsx
│   │   │   └── PageTitle.tsx
│   │   ├── common/
│   │   │   ├── Button.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Modal.tsx
│   │   │   ├── Badge.tsx
│   │   │   ├── Spinner.tsx
│   │   │   ├── EmptyState.tsx
│   │   │   └── ExportButton.tsx
│   │   ├── home/
│   │   │   ├── MetricCard.tsx
│   │   │   ├── PipelineBar.tsx
│   │   │   └── ActivityFeed.tsx
│   │   └── leads/
│   │       ├── LeadTable.tsx
│   │       ├── LeadFilters.tsx
│   │       └── LeadDetail.tsx
```

```
├── LeadRow.tsx
├── BulkActions.tsx
├── whatsapp/
│   ├── QueueItem.tsx
│   ├── QueueList.tsx
│   └── QueueStats.tsx
├── followups/
│   ├── FollowUpList.tsx
│   ├── FollowUpItem.tsx
│   ├── FollowUpTabs.tsx
│   └── RescheduleModal.tsx
├── reports/
│   ├── DateRangePicker.tsx
│   ├── OutreachChart.tsx
│   ├── ResponseChart.tsx
│   ├── StatusPieChart.tsx
│   └── TemplateTable.tsx
├── pages/
│   ├── Home.tsx
│   ├── Leads.tsx
│   ├── LeadDetail.tsx
│   ├── WhatsAppQueue.tsx
│   ├── FollowUps.tsx
│   └── Reports.tsx
├── lib/
│   ├── api.ts
│   ├── types.ts
│   └── export.ts
├── hooks/
│   ├── useLeads.ts
│   ├── useWhatsAppQueue.ts
│   ├── useFollowUps.ts
│   ├── useMetrics.ts
│   └── useRefresh.ts
├── context/
│   └── AppContext.tsx
├── App.tsx
├── main.tsx
├── index.css
├── .env
├── package.json
├── vite.config.ts
├── tsconfig.json
├── tailwind.config.js
└── postcss.config.js
```

Implementation Phases

Phase 1: Foundation (Files: 1-15)

1.1 Configuration Files

- `package.json` — dependencies and scripts
- `vite.config.ts` — Vite configuration with React plugin
- `tsconfig.json` — TypeScript config with path aliases
- `tailwind.config.js` — Tailwind with custom colors
- `postcss.config.js` — PostCSS for Tailwind
- `index.html` — HTML entry point
- `.env` — Environment variables (API URL, token, archive days)

1.2 Core Files

- `src/index.css` — Tailwind directives + custom styles
 - `src/main.tsx` — React entry point
 - `src/App.tsx` — Router setup
-

Phase 2: Core Infrastructure (Files: 16-22)

2.1 Types (`src/lib/types.ts`)

```
// Lead, FollowUp, WhatsAppQueueItem, Metrics, ApiResponse, etc.  
// All interfaces matching engine API schemas
```

2.2 API Client (`src/lib/api.ts`)

```
// Axios instance with base URL and auth header  
// Functions: getLeads, getLead, markWhatsAppSent, getMetrics, etc.  
// Error handling and response typing
```

2.3 Export Utilities (`src/lib/export.ts`)

```
// exportToCSV(data, filename)  
// exportToExcel(data, filename)  
// Uses PapaParse and XLSX libraries
```

2.4 React Context (`src/context/AppContext.tsx`)

```
// Shared state: lastSyncTime, refreshKey  
// Sync function that triggers data refresh across pages
```

2.5 Custom Hooks

- `src/hooks/useRefresh.ts` — Refresh trigger for sync button
 - `src/hooks/useLeads.ts` — Fetch and manage leads with filters
 - `src/hooks/useWhatsAppQueue.ts` — Queue management
 - `src/hooks/useFollowUps.ts` — Follow-up fetching
 - `src/hooks/useMetrics.ts` — Dashboard metrics
-

Phase 3: Layout Components (Files: 23-26)

3.1 Layout (`src/components/layout/Layout.tsx`)

- Main wrapper with sidebar + content area
- Responsive structure

3.2 Sidebar (`src/components/layout/Sidebar.tsx`)

- Navigation items: Home, Leads, WhatsApp Queue, Follow-ups, Reports
- Active state highlighting
- Logo/branding

3.3 Header (`src/components/layout/Header.tsx`)

- Page title
- Sync button with last sync time
- User info (optional)

3.4 PageTitle (`src/components/layout/PageTitle.tsx`)

- Reusable title component with optional actions
-

Phase 4: Common Components (Files: 27-34)

4.1 Button (`src/components/common/Button.tsx`)

- Variants: primary, secondary, danger, ghost
- Sizes: sm, md, lg
- Loading state

4.2 Card (`src/components/common/Card.tsx`)

- Wrapper for content sections
- Optional title and actions

4.3 Modal (`src/components/common/Modal.tsx`)

- Overlay modal for confirmations and forms
- Close button and footer actions

4.4 Badge (`src/components/common/Badge.tsx`)

- Status indicators
- Variants: success, warning, danger, info

4.5 Spinner ([src/components/common/Spinner.tsx](#))

- Loading indicator
- Sizes: sm, md, lg

4.6 EmptyState ([src/components/common/EmptyState.tsx](#))

- No data placeholder
- Optional action button

4.7 ExportButton ([src/components/common/ExportButton.tsx](#))

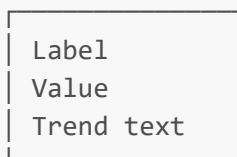
- Dropdown with CSV/Excel options
- Calls export utilities

Phase 5: Home Page (Files: 35-45)

5.1 Home Page ([src/pages/Home.tsx](#))

- Grid layout for metric cards
- Pipeline bar chart
- Activity feed
- Quick action buttons

5.2 MetricCard ([src/components/home/MetricCard.tsx](#))



- Props: title, value, trend, icon, alert

5.3 PipelineBar ([src/components/home/PipelineBar.tsx](#))

- Horizontal stacked bar
- Stage labels with counts
- Click to filter leads by stage

5.4 ActivityFeed ([src/components/home/ActivityFeed.tsx](#))

- List of recent actions
- Timestamps and actor names

- Links to related leads
-

Phase 6: Leads Page (Files: 46-55)

6.1 Leads Page ([src/pages/Leads.tsx](#))

- Filter bar at top
- Table with checkboxes for bulk selection
- Bulk actions bar (shown when items selected)
- Pagination
- Export button

6.2 LeadFilters ([src/components/leads/LeadFilters.tsx](#))

- Status dropdown
- City dropdown
- Industry dropdown
- Search input
- Apply/Reset buttons

6.3 LeadTable ([src/components/leads/LeadTable.tsx](#))

- Sortable columns
- Checkbox for selection
- Row click → detail view
- Columns: Company, Status, Employees, City, Industry, WhatsApp Status

6.4 LeadRow ([src/components/leads/LeadRow.tsx](#))

- Single row component
- Selection checkbox
- Status badge
- WhatsApp indicator

6.5 BulkActions ([src/components/leads/BulkActions.tsx](#))

- Shown when 1+ leads selected
- Actions: Mark Sent, Mark Unsent, Archive
- Selection count badge

6.6 Lead Detail Page ([src/pages/LeadDetail.tsx](#))

- Full lead information
- Activity history timeline
- Action buttons: Open WhatsApp, Update Status, Add Note

6.7 LeadDetail Component ([src/components/leads/LeadDetail.tsx](#))

- Company info section
 - Contact details
 - Status management
 - Notes/history
-

Phase 7: WhatsApp Queue Page (Files: 56-60)

7.1 WhatsApp Queue Page ([src/pages/WhatsAppQueue.tsx](#))

- Queue stats at top
- Sort dropdown
- List of queue items
- Manual refresh button

7.2 QueueList ([src/components/whatsapp/QueueList.tsx](#))

- Maps over queue items
- Pagination or infinite scroll

7.3 QueueItem ([src/components/whatsapp/QueueItem.tsx](#))

```
Company Name
Employees | City | Industry
Template: [Template Name]

[Open WhatsApp] [Skip] [Mark as Sent]
```

- Visual state: READY (default), OPENED (yellow border), SENT (green checkmark)
- Workflow:
 1. Click "Open WhatsApp" → calls `POST /leads/{id}/whatsapp/opened` → opens wa.me link
 2. User manually sends message
 3. Click "Mark as Sent" → calls `POST /leads/{id}/whatsapp/mark-sent`

7.4 QueueStats ([src/components/whatsapp/QueueStats.tsx](#))

```
Total: 23 | Processed: 5 | Remaining: 18
```

- Progress bar
-

Phase 8: Follow-ups Page (Files: 61-66)

8.1 Follow-ups Page ([src/pages/FollowUps.tsx](#))

- Tabs: Today, Overdue, Upcoming, All
- Count badges per tab
- List of follow-up items

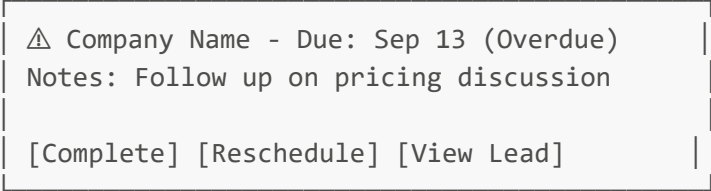
8.2 FollowUpTabs ([src/components/followups/FollowUpTabs.tsx](#))

- Tab buttons with counts
- Active tab highlighting

8.3 FollowUpList ([src/components/followups/FollowUpList.tsx](#))

- Groups by due date or overdue status
- Maps to FollowUpItem components

8.4 FollowUpItem ([src/components/followups/FollowUpItem.tsx](#))



⚠ Company Name - Due: Sep 13 (Overdue)
Notes: Follow up on pricing discussion
[Complete] [Reschedule] [View Lead]

- Color coding: Overdue (red), Today (yellow), Upcoming (gray)
- Complete → marks as done, hides from list
- Reschedule → opens modal

8.5 RescheduleModal ([src/components/followups/RescheduleModal.tsx](#))

- Date picker
- Reason textarea
- Save/Cancel buttons
- Calls `POST /api/v1/followups/{id}/reschedule`

Phase 9: Reports Page (Files: 67-72)

9.1 Reports Page ([src/pages/Reports.tsx](#))

- Date range picker at top
- Grid of charts
- Export button

9.2 DateRangePicker ([src/components/reports/DateRangePicker.tsx](#))

- Presets: Last 7 days, Last 30 days, This Month, Custom
- Custom range with start/end date pickers

9.3 OutreachChart ([src/components/reports/OutreachChart.tsx](#))

- Bar chart: Emails + WhatsApp by day
- X-axis: Date
- Y-axis: Count
- Legend

9.4 ResponseChart ([src/components/reports/ResponseChart.tsx](#))

- Line chart: Response rate trend
- X-axis: Date
- Y-axis: Percentage

9.5 StatusPieChart ([src/components/reports/StatusPieChart.tsx](#))

- Pie chart: Lead status distribution
- Legend with counts

9.6 TemplateTable ([src/components/reports/TemplateTable.tsx](#))

- Table: Template name, Sent count, Response count, Response rate
- Sortable by response rate
- Highlights best performers

API Endpoints (Engine API)

Dashboard

```
GET /api/v1/dashboard/metrics
```

Returns: emails sent, WhatsApp queue count, follow-ups due, replies today, pipeline breakdown

Leads

```
GET /api/v1/leads?status=&city=&industry=&search=&page=&limit=
GET /api/v1/leads/{lead_id}
POST /api/v1/leads/{lead_id}/whatsapp/opened
POST /api/v1/leads/{lead_id}/whatsapp/mark-sent
POST /api/v1/leads/bulk/mark-sent
POST /api/v1/leads/bulk/mark-unsent
POST /api/v1/leads/bulk/archive
```

WhatsApp Queue

```
GET /api/v1/outreach/whatsapp-queue
```

Returns: prioritized list with lead_id, whatsapp_url, status, template

Follow-ups

```
GET /api/v1/followups?status=today|overdue|upcoming
POST /api/v1/followups
POST /api/v1/followups/{followup_id}/complete
POST /api/v1/followups/{followup_id}/reschedule
```

Reports

```
GET /api/v1/reports/outreach?start_date=&end_date=
GET /api/v1/reports/templates
GET /api/v1/activity?limit=50
```

Sync

```
POST /api/v1/sync
```

Triggers engine data refresh

TypeScript Interfaces ([src/lib/types.ts](#))

```
// Lead
interface Lead {
  id: string;
  company_name: string;
  status: 'new' | 'contacted' | 'responded' | 'qualified' | 'won' | 'lost';
  employees: number;
  city: string;
  industry: string;
  linkedin_url?: string;
  phone?: string;
  whatsapp_number?: string;
  whatsapp_status: 'pending' | 'opened' | 'sent';
  created_at: string;
  updated_at: string;
}

// Follow-up
```

```
interface FollowUp {
  id: string;
  lead_id: string;
  company_name: string;
  due_date: string;
  status: 'pending' | 'completed' | 'overdue';
  notes?: string;
  created_at: string;
}

// WhatsApp Queue Item
interface WhatsAppQueueItem {
  lead_id: string;
  company_name: string;
  employees: number;
  city: string;
  industry: string;
  template: string;
  whatsapp_url: string;
  status: 'READY' | 'OPENED' | 'SENT';
}

// Dashboard Metrics
interface DashboardMetrics {
  emails_sent_today: number;
  whatsapp_queue_count: number;
  followups_due_today: number;
  replies_today: number;
  pipeline: {
    new: number;
    contacted: number;
    responded: number;
    qualified: number;
    won: number;
  };
  last_sync: string;
}

// Activity
interface Activity {
  id: string;
  type: 'email_sent' | 'whatsapp_opened' | 'whatsapp_sent' | 'follow_up_created' |
  'lead_created';
  lead_id?: string;
  company_name?: string;
  timestamp: string;
  details?: string;
}

// API Filters
interface LeadFilters {
  status?: string;
  city?: string;
  industry?: string;
}
```

```
    search?: string;
    page?: number;
    limit?: number;
  }

// API Response
interface ApiResponse<T> {
  data: T;
  total?: number;
  page?: number;
  limit?: number;
  has_more?: boolean;
}
```

Environment Variables

```
VITE_ENGINE_API_URL=http://localhost:8000
VITE_ENGINE_API_TOKEN=your-api-token-here
VITE_LEAD_ARCHIVE_DAYS=7
```

Development Workflow

Session 1: Foundation

1. Create all config files (package.json, vite.config, tsconfig, tailwind)
2. Create index.css with Tailwind
3. Create main.tsx and App.tsx with basic router
4. Verify project runs with `npm run dev`

Session 2: Core Infrastructure

1. Create types.ts with all interfaces
2. Create api.ts with Axios setup and all API functions
3. Create export.ts utilities
4. Create AppContext for shared state
5. Create useRefresh hook

Session 3: Layout

1. Create Layout, Sidebar, Header components
2. Wrap all pages in Layout
3. Implement navigation between pages
4. Add sync button to header

Session 4: Common Components

1. Create Button, Card, Modal, Badge, Spinner, EmptyState
2. Create ExportButton component
3. Test components in isolation

Session 5: Home Page

1. Create Home page with metric cards
2. Create MetricCard component
3. Create PipelineBar component
4. Create ActivityFeed component
5. Connect to `/api/v1/dashboard/metrics`

Session 6: Leads Page (Part 1)

1. Create Leads page structure
2. Create LeadFilters component
3. Create LeadTable component
4. Create LeadRow component
5. Connect to `/api/v1/leads`

Session 7: Leads Page (Part 2)

1. Create BulkActions component
2. Implement bulk selection logic
3. Connect bulk actions to API
4. Add ExportButton with CSV/Excel export
5. Create Lead Detail page

Session 8: WhatsApp Queue

1. Create WhatsAppQueue page
2. Create QueueList and QueueItem components
3. Implement "Open WhatsApp" → "Mark as Sent" workflow
4. Create QueueStats component

Session 9: Follow-ups

1. Create FollowUps page with tabs
2. Create FollowUpList and FollowUpItem components
3. Create RescheduleModal component
4. Connect to follow-ups API

Session 10: Reports

1. Create Reports page
2. Create DateRangePicker component
3. Create OutreachChart, ResponseChart, StatusPieChart
4. Create TemplateTable component
5. Add export functionality

Session 11: Polish & Testing

1. Error handling and loading states
 2. Responsive adjustments
 3. Test all workflows end-to-end
 4. Performance optimization
-

Important Notes

WhatsApp Flow

1. **Open WhatsApp** does NOT mark as sent
2. User manually sends message in WhatsApp app
3. User returns and clicks **Mark as Sent**
4. Engine updates CRM and schedules follow-up

Sync Behavior

- No real-time websockets
- Manual refresh only (Sync button or browser refresh)
- Each refresh fetches fresh data from API

Lead Archival

- Leads archived after 7 days (configurable via env)
- Archived leads not shown in default views
- Archive is soft delete (data retained)

Export

- CSV: Full lead data with all fields
- Excel: Formatted spreadsheet with multiple sheets option
- Export respects current filters

Bulk Operations

- Bulk mark sent: Updates WhatsApp status for multiple leads
 - Bulk mark unsent: Reverts status to pending
 - Bulk archive: Soft deletes selected leads
-

Dependencies to Install

```
npm install
```

Will install:

- react, react-dom

- react-router-dom
 - axios
 - recharts
 - date-fns
 - papaparse
 - xlsx
 - clsx
 - tailwindcss, autoprefixer, postcss
-

Running the Dashboard

```
# Development
npm run dev

# Build for production
npm run build

# Preview production build
npm run preview
```

Next Steps for Tonight

1. Review this plan and clarify any questions
 2. Set up the engine API endpoints (separate task)
 3. Start with Session 1 (Foundation) when ready to code
 4. Test each phase before moving to the next
-

Key Decisions Made

- **Tech:** React + Vite (confirmed)
- **Auth:** Single shared API token
- **Archival:** 7 days
- **Export:** CSV + Excel
- **Bulk ops:** Yes (mark sent, mark unsent, archive)
- **Sync:** Manual refresh only
- **Device:** Desktop-only (local access)
- **Users:** 3-4 team members with same token